COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	09 / 17 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9 **Trees**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

CODE

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct TreeNode {
    char data;
    TreeNode* leftChild;
    TreeNode* nextSibling;

    TreeNode(char val) {
        data = val;
        leftChild = nullptr;
        nextSibling = nullptr;
    }
};

void addChild(TreeNode* parent, TreeNode* child) {
    if (!parent || !child) return;

    if (!parent->leftChild) {
        parent->leftChild = child;
    } else {
        TreeNode* current = parent->leftChild;
        while (current->nextSibling) {
            current = current->nextSibling;
        }
        current->nextSibling = child;
    }
}
```

```cpp
int findDepth(TreeNode* root, char target, int currentDepth = 0) {
    if (!root) return -1;
    if (root->data == target) return currentDepth;

    int childDepth = findDepth(root->leftChild, target, currentDepth + 1);
    if (childDepth != -1) return childDepth;

    return findDepth(root->nextSibling, target, currentDepth);
}

int findHeight(TreeNode* node) {
    if (!node) return -1;

    int maxHeight = 0;

    TreeNode* child = node->leftChild;

    while (child) {
        maxHeight = max(maxHeight, 1 + findHeight(child));
        child = child->nextSibling;
    }

    return maxHeight;
}

int main() {

    TreeNode* A = new TreeNode('A');
    TreeNode* B = new TreeNode('B');
    TreeNode* C = new TreeNode('C');
    TreeNode* D = new TreeNode('D');
    TreeNode* E = new TreeNode('E');
    TreeNode* F = new TreeNode('F');
    TreeNode* G = new TreeNode('G');
    TreeNode* H = new TreeNode('H');
    TreeNode* I = new TreeNode('I');
    TreeNode* J = new TreeNode('J');
    TreeNode* K = new TreeNode('K');
    TreeNode* L = new TreeNode('L');
    TreeNode* M = new TreeNode('M');
    TreeNode* N = new TreeNode('N');
    TreeNode* P = new TreeNode('P');
    TreeNode* Q = new TreeNode('Q');

    addChild(A, B); addChild(A, C); addChild(A, D); addChild(A, E); addChild(A, F); addChild(A, G);
    addChild(D, H);
    addChild(E, I); addChild(E, J);
    addChild(F, K); addChild(F, L); addChild(F, M);
    addChild(G, N);
    addChild(J, P); addChild(J, Q);

    vector<char> nodes = {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'P', 'Q'};

    cout << "Node\tHeight\tDepth" << endl;
    cout << "                    " << endl;

    for (char c : nodes) {
        TreeNode* targetNode = findNode(A, c);
        int h = findHeight(targetNode);
        int d = findDepth(A, c);
        cout << c << "\t" << h << "\t" << d << endl;
    }

    return 0;
}
```

```
#ifndef GENERAL_TREE_H
#define GENERAL_TREE_H

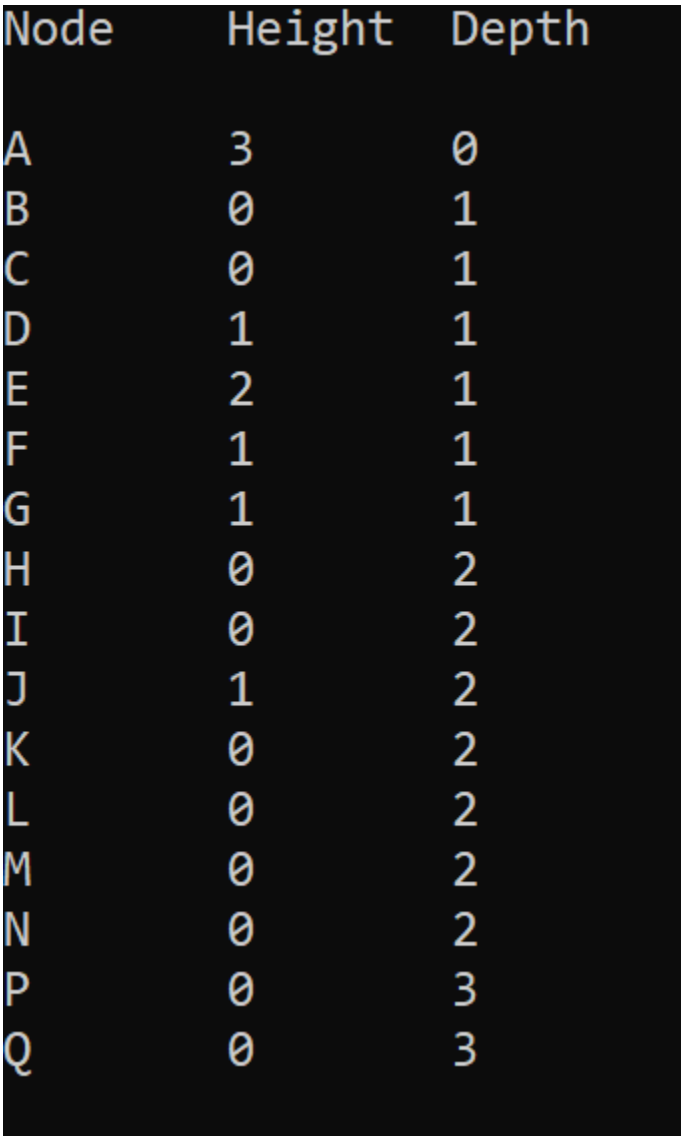
#include <algorithm>

struct TreeNode {
    char data;
    TreeNode* leftChild;
    TreeNode* nextSibling;

    TreeNode(char val) {
        data = val;
        leftChild = nullptr;
        nextSibling = nullptr;
    }
};

void addChild(TreeNode* parent, TreeNode* child);
int findDepth(TreeNode* root, char target, int currentDepth = 0);
int findHeight(TreeNode* node);
TreeNode* findNode(TreeNode* root, char target);

#endif
```

OUTPUT	
OBSERVATION	The code defines a template class inside the header. It ensures the structure is highly reusable, allowing the tree to accept integers, and float point numbers.

Node	Height	Dept
A	3	0
B	0	1
C	0	1

D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Code

```cpp
#include <iostream>
#include <string>
#include <vector>
#include "BST.h"

using namespace std;

BSTNode* insertNode(BSTNode* node, int val) {
    if (!node) {
        return new BSTNode(val);
    }
    if (val < node->data) {
        node->left = insertNode(node->left, val);
    } else if (val > node->data) {
        node->right = insertNode(node->right, val);
    }
    return node;
}

void preorderTraversal(BSTNode* node) {
    if (!node) return;
    cout << node->data << " ";
    preorderTraversal(node->left);
    preorderTraversal(node->right);
}

void inorderTraversal(BSTNode* node) {
    if (!node) return;
    inorderTraversal(node->left);
    cout << node->data << " ";
    inorderTraversal(node->right);
}
```

```
void postorderTraversal(BSTNode* node) {
    if (!node) return;
    postorderTraversal(node->left);
    postorderTraversal(node->right);
    cout << node->data << " ";
}

bool searchBST(BSTNode* node, int key) {
    if (!node) return false;
    if (node->data == key) return true;
    if (key < node->data) return searchBST(node->left, key);
    return searchBST(node->right, key);
}

void findData(BSTNode* root, string choice, int key) {

    for (auto &c : choice) c = toupper(c);

    bool found = searchBST(root, key);

    if (found) {
        cout << "\"" << key << "\" was found!" << endl;
    }
}

int main() {
    BSTNode* root = nullptr;

    vector<int> initialValues = {50, 30, 20, 40, 70, 60, 80};
    for (int val : initialValues) {
        root = insertNode(root, val);
    }
```

```
    cout << "Pre-order:  "; preorderTraversal(root);  cout << endl;
    cout << "In-order:   "; inorderTraversal(root);   cout << endl;
    cout << "Post-order: "; postorderTraversal(root); cout << endl << endl;

    findData(root, "INORDER", 40);
    cout << endl;

    root = insertNode(root, 45);

    findData(root, "PREORDER", 45);

    return 0;
}
```

```
#ifndef BST_H
#define BST_H

#include <iostream>
#include <string>


struct BSTNode {
    int data;
    BSTNode* left;
    BSTNode* right;

    BSTNode(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

BSTNode* insertNode(BSTNode* node, int val);
void preorderTraversal(BSTNode* node);
void inorderTraversal(BSTNode* node);
void postorderTraversal(BSTNode* node);
void findData(BSTNode* root, std::string choice, int key);

#endif
```

Output:

```
Pre-order:  50 30 20 40 70 60 80
In-order:   20 30 40 50 60 70 80
Post-order: 20 40 30 60 80 70 50

"40" was found!

"45" was found!

--------------------------------
Process exited after 0.0231 seconds with return value 0
Press any key to continue . . .
```

Pre-order	**50, 30, 20, 40, 70, 60, 80**
Post-order	**20, 30, 40, 50, 60, 70, 80**
In-order	**20, 40, 30, 60, 80, 70, 50**

Based on the code, it utilizes recursion to navigate through the tree and uses the execution stack to move downward through node addresses and safely backtrack up parent branches once nullptr is reached. It immediately prints the data of whatever node was positioned.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```
#include <iostream>
#include "BST.h"

int main(){

BinarySearchTree<int>* tree = new BinarySearchTree<int>(2);

    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);

    std::cout<<"\nPre Order output:";
    tree->preOrder();
    std::cout<<std::endl;

    std::cout<<"\nIn Order output:";
    tree->inOrder();
    std::cout<<std::endl;

        std::cout<<"\nPost Order output:";
    tree->postOrder();
    std::cout<<std::endl;
    return 0;



};
```

```
#ifndef BST_H
#define BST_H

#include <iostream>

template<typename T>
class BinarySearchTree {

private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:
    BinarySearchTree(T value);
    ~BinarySearchTree();
    void insert(T value);
    void preOrder();
    void inOrder();
    void postOrder();
};

template<typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}
```

```
}

template<typename T>
BinarySearchTree<T>::~BinarySearchTree() {
    delete left;
    delete right;
}

template<typename T>
void BinarySearchTree<T>::insert(T value) {
    if (value < key) {
        if (left == nullptr) {
            left = new BinarySearchTree<T>(value);
        } else {
            left->insert(value);
        }
    }
    else if (value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree<T>(value);
        } else {
            right->insert(value);
        }
    }
}
```

```cpp
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::inOrder(){
    if (left != nullptr) {
        left->inOrder();
    }

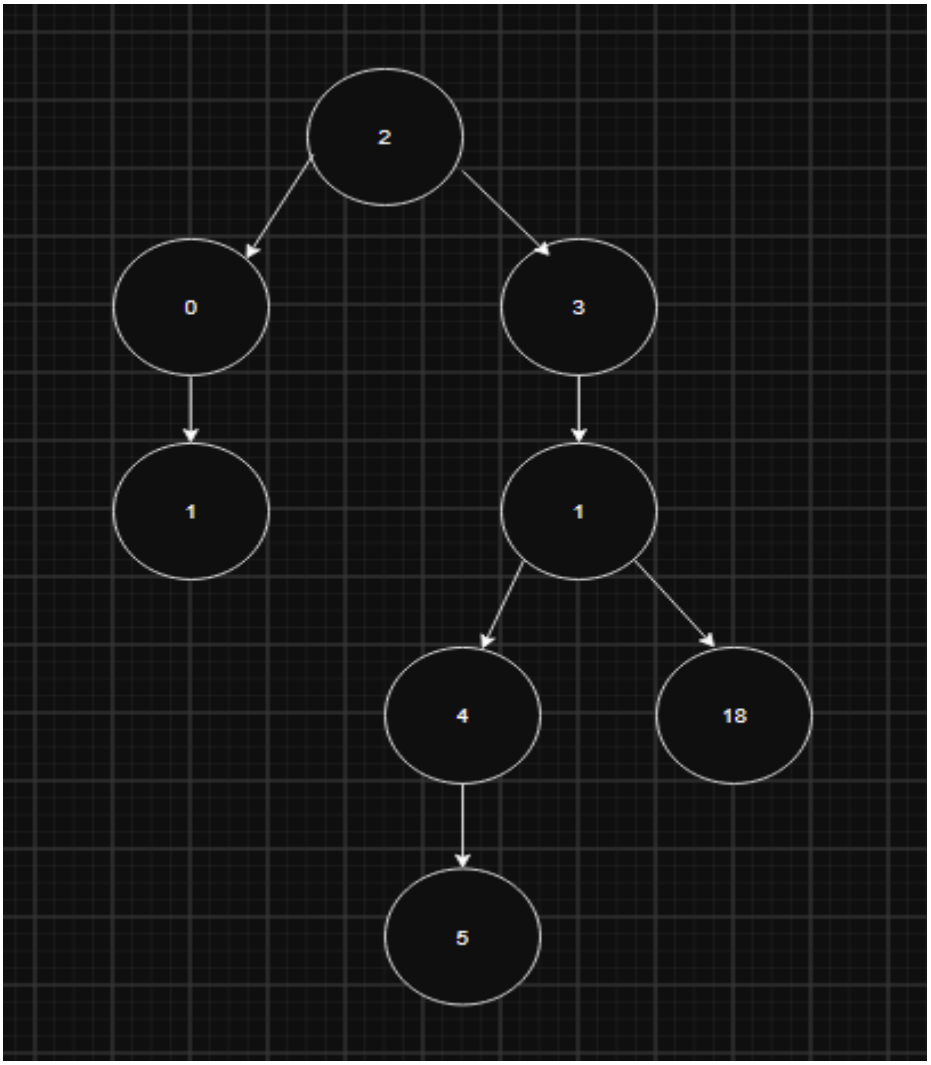
    std::cout << key << " ";

    if (right != nullptr) {
        right->inOrder();
    }
}

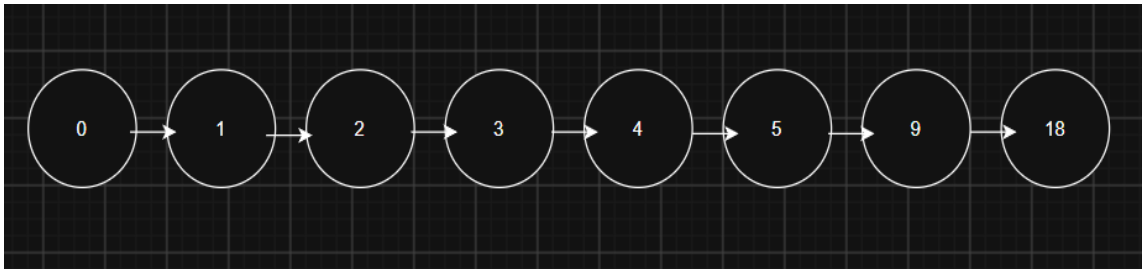
template<typename T>
void BinarySearchTree<T>::postOrder(){
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}

#endif
```

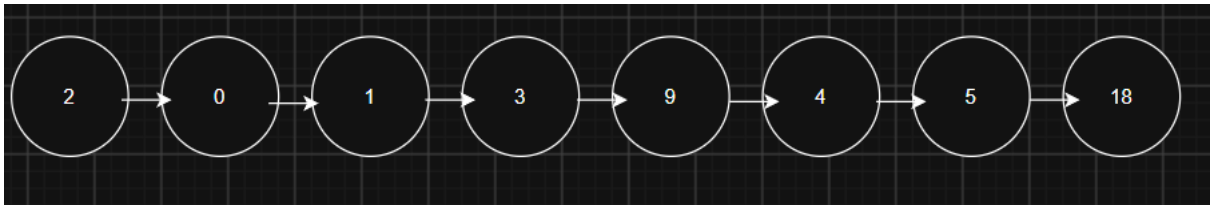
Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on The diagram.



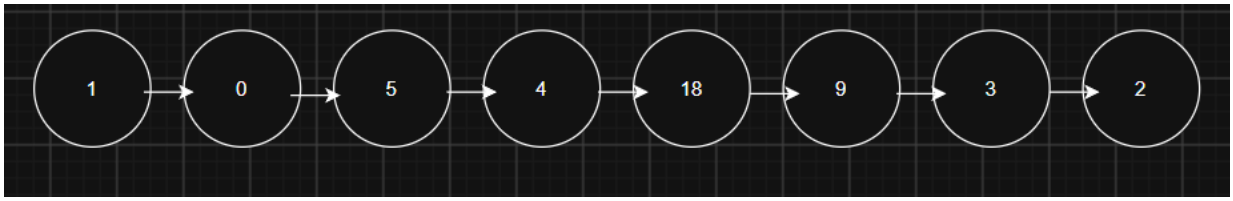
In order traversal



Pre - Order traversal



Post Order traversal



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
Pre Order output:2 0 1 3 9 4 5 18

In Order output:0 1 2 3 4 5 9 18

Post Order output:1 0 5 4 18 9 3 2
```

- In order traversal retrieve elements in sorted order from lowest to highest value.
- Pre order records the hierarchy layers first before moving downward pathways
- Post order evaluates all bottom level leaf nodes before moving upward to process parent structure.

Pre- order traversal

```
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}
```

```
Pre Order output:2 0 1 3 9 4 5 18
```

Post Order traversal

```
template<typename T>
void BinarySearchTree<T>::postOrder(){
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}
```

```
Post Order output:1 0 5 4 18 9 3 2
```

Conclusion / Learning analysis:

Observing how the search tree structure controls the data organization by applying the rule where smaller values are routed left and larger values are routed right. Random sequence of inputs automatically builds a balanced searchable hierarchy. As a student, learning tree traversal made abstract data structure concepts more practical. Seeing how moving print statements around the recursive alters the output sequence helped me understand how it navigates memory.

Based on my performance when completing this activity, I managed to implement and debug the code for 3 traversals and their outputs. But in all honesty, I still struggle on creating a binary search tree on my own but i will keep on practicing.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.